Ministerul Educaţiei şi Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică şi Microelectronică

Laboratory work:

Assembly Language — Familiarization, Installation, and Programming with NASM

Elaborated:
st. gr. FAF-243 Kushnirenko Ecaterina

Verified:
asist. univ. Fiştic Cristofor

Chişinău - 2026

Contents

1 INTRODUCTION

1.1 Brief Explanation of Assembly Language

Assembly Language is one of the lowest-level programming languages available to developers, sitting just one abstraction layer above raw machine code. Unlike high-level languages such as Python or Java, Assembly provides direct control over the computer's hardware — the programmer interacts with the CPU's registers, memory addresses, and system calls at the most fundamental level. Each Assembly instruction corresponds closely to a single machine code operation that the processor executes.

Assembly uses *mnemonics* — short symbolic names such as `MOV`, `ADD`, `SUB`, `JMP` — to represent binary CPU instructions. These mnemonics are translated into executable machine code by an *assembler*. The resulting programs are extremely efficient in terms of speed and memory usage, but require careful attention to detail, as there are no safety nets like type checking, garbage collection, or automatic memory management.

1.2 Importance of Low-Level Programming

Understanding low-level programming is essential for computer science students because it reveals how software truly interacts with hardware. Concepts such as memory management, CPU architecture, instruction pipelines, register allocation, and system calls become tangible rather than abstract.

This knowledge is critical for several fields:

- **Embedded systems** — programming microcontrollers with limited resources
- **Operating system development** — writing kernels, drivers, and bootloaders
- **Reverse engineering and cybersecurity** — analysing malware, exploits, and binaries
- **Performance optimisation** — writing hand-tuned code for critical sections
- **Computer architecture** — understanding how CPUs execute instructions

2 PURPOSE OF THE REPORT

The purpose of this report is to document the complete process of learning and working with Assembly Language through a hands-on laboratory exercise. Specifically, this report covers:

- Familiarisation with core Assembly concepts (registers, memory, instructions, syntax)
- Installation and configuration of the chosen development environment
- Writing, running, and debugging three original Assembly programs
- Line-by-line commentary explaining every instruction

2.1 Chosen Application and Justification

The application chosen for this laboratory is **SASM (SimpleASM IDE)** paired with the **NASM (Netwide Assembler)** backend. SASM was selected for the following reasons:

Table 1: Reasons for choosing SASM with NASM

Criterion	**Justification**
Integrated environment	SASM combines a code editor, assembler, and GDB debugger in a single Qt-based application, eliminating the need to switch between terminal windows
Beginner-friendly	Includes syntax highlighting, built-in output console, and a register/memory viewer for step-by-step visualisation
Cross-platform	Runs on Linux, Windows, and macOS; available on Arch Linux through the AUR (Arch User Repository)
Intel syntax	NASM uses Intel syntax, which is more intuitive and readable than AT&T syntax used by GAS; most educational materials use Intel syntax

3 FAMILIARIZATION WITH ASSEMBLY BASICS

3.1 Program Structure

Every NASM Assembly program is divided into three main sections:

Table 2: Assembly program sections

Section	**Directive**	**Purpose**
Initialised data	`section .data`	Holds variables and constants with predefined values (strings, numbers)
Uninitialised data	`section .bss`	Reserves space for variables that will be filled at runtime (input buffers)
Executable code	`section .text`	Contains the actual program instructions; entry point declared with `global main`

3.2 Registers

Registers are small, fast storage locations inside the CPU — the primary workspace for Assembly programs. On x86 (32-bit) systems, the main general-purpose registers are:

Table 3: x86 general-purpose registers

Register	**Name**	**Primary Purpose**
EAX	Accumulator	Arithmetic operations, return values, syscall numbers
EBX	Base	Base addressing, syscall argument 1
ECX	Counter	Loop counter, syscall argument 2
EDX	Data	I/O operations, division remainder, syscall argument 3
ESI	Source Index	Pointer to source data in string/memory operations
EDI	Destination Index	Pointer to destination in string/memory operations
ESP	Stack Pointer	Points to the top of the stack (managed automatically)
EBP	Base Pointer	Points to base of current stack frame in functions

Each 32-bit register (e.g., EAX) can also be accessed as a 16-bit register (AX) or as two 8-bit halves (AH for the high byte and AL for the low byte). This flexibility allows efficient handling of data of different sizes.

3.3 Memory and Addressing

Assembly programs interact with memory using addresses. Square brackets denote memory access: `[address]` reads from or writes to the memory at that location.

Data declaration directives:

- `db` — define byte (1 byte)
- `dw` — define word (2 bytes)
- `dd` — define double word (4 bytes)
- `resb`, `resw`, `resd` — reserve uninitialised bytes, words, or double words
- `equ` — define a compile-time constant

3.4 Key Instructions

Table 4: Commonly used x86 NASM instructions

Instruction	Description
`MOV dest, src`	Copies data from source to destination
`ADD dest, src`	Adds source to destination, stores result in destination
`SUB dest, src`	Subtracts source from destination
`IMUL dest, src`	Signed multiplication
`IDIV src`	Signed division: EAX = quotient, EDX = remainder
`CMP op1, op2`	Compares two values by subtracting (sets CPU flags)
`JMP label`	Unconditional jump to a label
`JE / JNE`	Jump if equal / not equal (based on flags from CMP)
`JG / JL / JGE`	Jump if greater / less / greater or equal
`PUSH value`	Pushes a value onto the stack
`POP dest`	Pops the top value from the stack into destination
`INT 0x80`	Triggers a Linux kernel system call interrupt
`XOR reg, reg`	Common idiom to set a register to zero efficiently
`INC / DEC`	Increment / decrement a value by 1

3.5 System Calls (Linux)

On 32-bit Linux, programs communicate with the operating system kernel through software interrupt `int 0x80`. Before triggering the interrupt, the program places the syscall number in EAX and arguments in EBX, ECX, EDX.

Table 5: Common Linux system calls

Syscall	EAX	Arguments
sys_exit	1	EBX = exit code
sys_read	3	EBX = fd, ECX = buffer, EDX = count
sys_write	4	EBX = fd, ECX = buffer, EDX = count

File descriptor 0 = standard input (keyboard), 1 = standard output (screen), 2 = standard error.

4 INSTALLATION AND SETUP

4.1 Description of the Selected Program

SASM (SimpleASM IDE) is an open-source, cross-platform integrated development environment designed specifically for learning Assembly Language. It was created by Dmitriy Manushin and provides a graphical interface built with the Qt framework. SASM supports multiple assemblers including NASM, MASM, GAS, and FASM. For this laboratory, NASM was chosen as the backend assembler.

4.2 System Compatibility

Table 6: System environment

Parameter	Value
Operating System	Arch Linux (rolling release, x86_64)
Kernel	Linux 6.x
Dependencies	NASM (assembler), GCC (linker), GDB (debugger), Qt libs
Package source	AUR for SASM; official repos for NASM, GCC, GDB

4.3 Steps of Installation

Step 1: Install an AUR helper (if not already present):

```
sudo pacman -S --needed git base-devel
git clone https://aur.archlinux.org/yay.git
cd yay
makepkg -si
```

Step 2: Install the assembler, linker, and debugger:

```
sudo pacman -S nasm gcc gdb
```

This installs NASM (translates `.asm` files into object code), GCC (links object files into executables), and GDB (the GNU Debugger used by SASM for step-by-step debugging).

Step 3: Install SASM from the AUR:

```
yay -S sasm
```

The AUR helper downloads the SASM source package, resolves Qt dependencies automatically, compiles the application, and installs it.

Step 4: Launch and verify:

```
sasm
```

4.4 Initial Configuration

After launching SASM for the first time, the following settings were verified:

- **Assembler:** Settings → Assembler → NASM selected
- **Assembler path:** `/usr/bin/nasm` (confirmed with `which nasm`)
- **Linker path:** `/usr/bin/gcc`
- **Debugger:** GDB configured automatically (Debug menu, F5 to start)
- **Mode:** 32-bit (x86) for compatibility with standard educational materials

5 SIMPLE PROGRAMS

Three programs were written in NASM Assembly using SASM. Each program demonstrates different core Assembly concepts. Every instruction is commented to explain its purpose.

5.1 Program 1: Temperature Converter (Celsius to Fahrenheit)

5.1.1 Concepts Demonstrated

User input via `sys_read`, ASCII-to-integer conversion, arithmetic operations (multiplication, division, addition), integer-to-ASCII conversion, and output via `sys_write`. The formula used is $F = C \times 9 \div 5 + 32$.

5.1.2 Source Code with Line-by-Line Comments

```
; ==========================================
; Program 1: Temperature Converter (C to F)
; Formula: F = C * 9 / 5 + 32
; ==========================================

section .data                       ; Initialised data section
    msg_input db "Enter Celsius: ", 0
                                    ; Prompt string, null-terminated
    msg_result db "Fahrenheit: ", 0
                                    ; Result label string
    newline db 10                   ; Newline character (ASCII 10)

section .bss                        ; Uninitialised data section
    input resb 10                  ; Reserve 10 bytes for user input
    celsius resd 1                 ; Reserve 4 bytes for Celsius value
    fahrenheit resd 1              ; Reserve 4 bytes for Fahrenheit
    output resb 10                 ; Reserve 10 bytes for output string

section .text                       ; Executable code section
    global main                     ; Declare entry point for linker

main:                               ; Program entry point
    ; --- Print the input prompt ---
    mov eax, 4                     ; syscall 4 = sys_write
    mov ebx, 1                     ; fd 1 = stdout (screen)
    mov ecx, msg_input             ; pointer to prompt string
    mov edx, 16                    ; number of bytes to write
    int 0x80                       ; invoke kernel

    ; --- Read user input from keyboard ---
    mov eax, 3                     ; syscall 3 = sys_read
    mov ebx, 0                     ; fd 0 = stdin (keyboard)
    mov ecx, input                 ; buffer to store typed text
    mov edx, 10                    ; max bytes to read
    int 0x80                       ; invoke kernel

    ; --- Convert ASCII string to integer ---
    mov esi, input                  ; ESI points to input buffer
    xor eax, eax                   ; clear EAX (result accumulator)
    xor ecx, ecx                   ; clear ECX (current character)

convert_loop:                       ; Process each ASCII digit
    mov cl, [esi]                  ; load 1 byte from buffer
```

```
    cmp cl, 10                  ; is it newline? (Enter key)
    je done_convert             ; yes -> conversion done
    cmp cl, '0'                 ; below ASCII '0'?
    jb done_convert             ; yes -> not a digit, stop
    cmp cl, '9'                 ; above ASCII '9'?
    ja done_convert             ; yes -> not a digit, stop
    sub cl, '0'                 ; convert ASCII to number (0-9)
    imul eax, eax, 10           ; shift result left by one decimal
    add eax, ecx                ; add new digit to result
    inc esi                     ; advance to next character
    jmp convert_loop            ; repeat for next digit

done_convert:                   ; EAX now holds integer value
    mov [celsius], eax          ; store Celsius in memory

    ; --- Apply formula: F = C * 9 / 5 + 32 ---
    imul eax, eax, 9            ; EAX = Celsius * 9
    cdq                         ; sign-extend EAX -> EDX:EAX
    mov ecx, 5                  ; divisor = 5
    idiv ecx                    ; EAX = (C*9)/5, EDX = remainder
    add eax, 32                 ; EAX = (C*9)/5 + 32 = Fahrenheit
    mov [fahrenheit], eax       ; store result in memory

    ; --- Convert integer to ASCII string ---
    mov eax, [fahrenheit]       ; load result into EAX
    mov edi, output             ; EDI points to output buffer
    add edi, 9                  ; start from end (reverse fill)
    mov byte [edi], 0           ; null-terminate

int_to_str:                     ; Extract digits right-to-left
    dec edi                     ; move pointer left
    xor edx, edx                ; clear EDX before division
    mov ecx, 10                 ; divisor = 10
    div ecx                     ; EAX=quotient, EDX=remainder
    add dl, '0'                 ; convert digit to ASCII
    mov [edi], dl               ; store in buffer
    test eax, eax               ; quotient zero?
    jnz int_to_str              ; no -> more digits remain

    ; --- Print result label ---
    mov eax, 4                  ; sys_write
    mov ebx, 1                  ; stdout
    mov ecx, msg_result         ; pointer to "Fahrenheit: "
    mov edx, 13                 ; length of label
    int 0x80                    ; invoke kernel

    ; --- Print the number ---
    mov eax, 4                  ; sys_write
    mov ebx, 1                  ; stdout
    mov ecx, edi                ; pointer to first digit
    mov edx, output             ; calculate length:
    add edx, 9                  ;   end of buffer
    sub edx, edi                ;   minus start = length
    int 0x80                    ; invoke kernel

    ; --- Print newline ---
    mov eax, 4                  ; sys_write
    mov ebx, 1                  ; stdout
    mov ecx, newline            ; pointer to newline byte
    mov edx, 1                  ; 1 byte
```

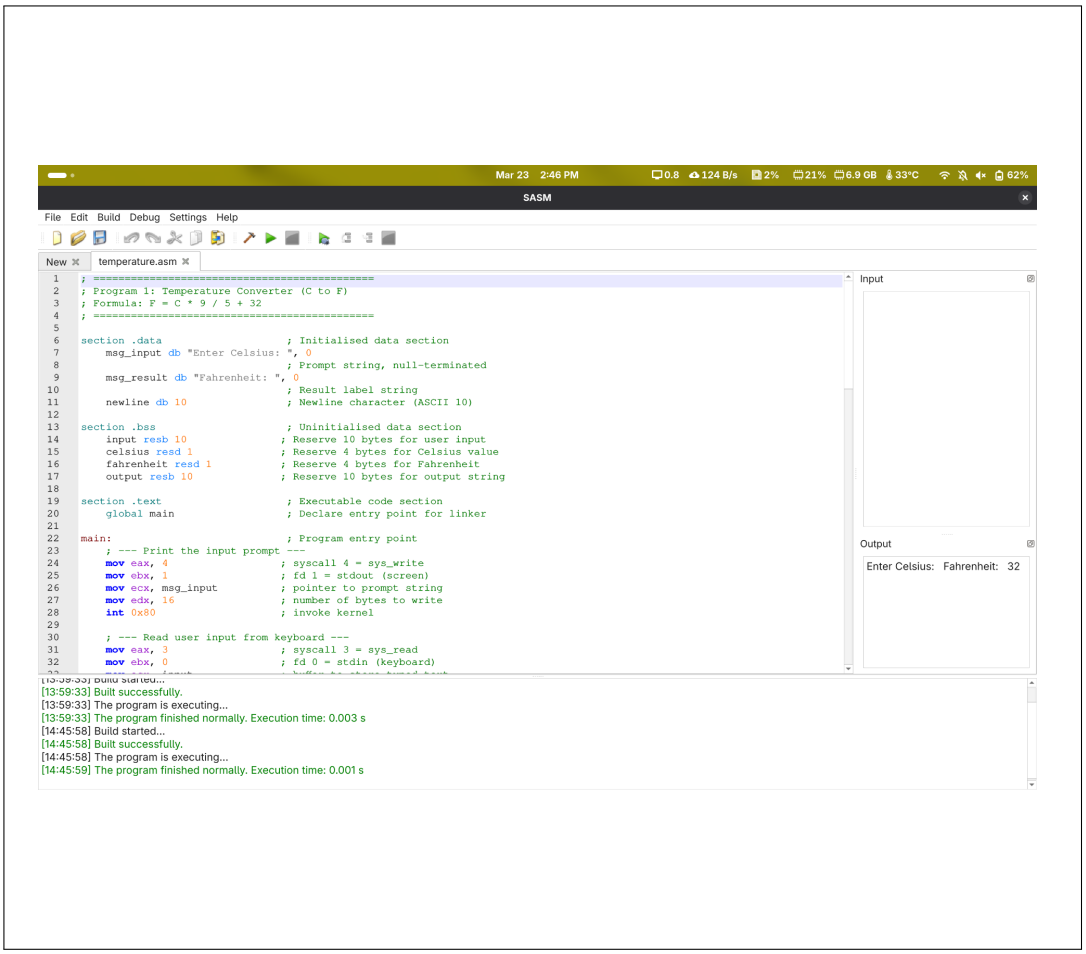
```
104     int 0x80                    ; invoke kernel
105
106     ; --- Exit program ---
107     mov eax, 1                  ; syscall 1 = sys_exit
108     xor ebx, ebx                ; exit code 0 (success)
109     int 0x80                    ; terminate process
```

Listing 1: temperature.asm — Celsius to Fahrenheit converter

5.1.3 Output Description

The program prompts the user to enter a temperature in Celsius. After typing a number and pressing Enter, it calculates the Fahrenheit equivalent and prints the result. For example, entering `100` produces: `Fahrenheit:  212`. Entering `0` produces: `Fahrenheit:  32`.

5.1.4 Screenshot: Code + Result



5.2 Program 2: Find the Maximum of Three Numbers

5.2.1 Concepts Demonstrated

Memory-defined variables with `dd`, comparison instruction `CMP`, conditional jumps (`JGE`), labels and control flow branching, and integer-to-string conversion for output.

5.2.2 Source Code with Line-by-Line Comments

```
; ==========================================
; Program 2: Find the Maximum of Three Numbers
; Compares three predefined values, prints the
; largest one to the screen
; ==========================================

section .data                       ; Initialised data section
    num1 dd 42                     ; First number (4 bytes)
    num2 dd 87                     ; Second number (4 bytes)
    num3 dd 65                     ; Third number (4 bytes)
    msg db "The maximum is: ", 0   ; Output label
    newline db 10                  ; Newline character

section .bss                        ; Uninitialised data section
    output resb 12                 ; Reserve 12 bytes for result

section .text                       ; Executable code section
    global main                     ; Declare entry point

main:                               ; Program entry point
    ; --- Load first number as initial max ---
    mov eax, [num1]                ; EAX = 42 (load from memory)

    ; --- Compare with second number ---
    cmp eax, [num2]                ; compare EAX with num2 (87)
    jge skip_num2                  ; if EAX >= num2, keep EAX
    mov eax, [num2]                ; else EAX = num2 (new max)

skip_num2:                          ; EAX = max(num1, num2) = 87
    ; --- Compare with third number ---
    cmp eax, [num3]                ; compare EAX with num3 (65)
    jge skip_num3                  ; if EAX >= num3, keep EAX
    mov eax, [num3]                ; else EAX = num3 (new max)

skip_num3:                          ; EAX = max of all three = 87
    ; --- Convert max to ASCII string ---
    mov edi, output                ; EDI -> output buffer
    add edi, 11                    ; start from end of buffer
    mov byte [edi], 0              ; null-terminate

convert_max:                        ; Extract digits right-to-left
    dec edi                       ; move pointer left 1 byte
    xor edx, edx                  ; clear EDX for division
    mov ecx, 10                   ; divisor = 10
    div ecx                       ; EAX=quotient, EDX=last digit
    add dl, '0'                   ; convert digit to ASCII
    mov [edi], dl                 ; store in buffer
    test eax, eax                 ; quotient zero?
    jnz convert_max               ; no -> continue extracting

    ; --- Print label ---
```

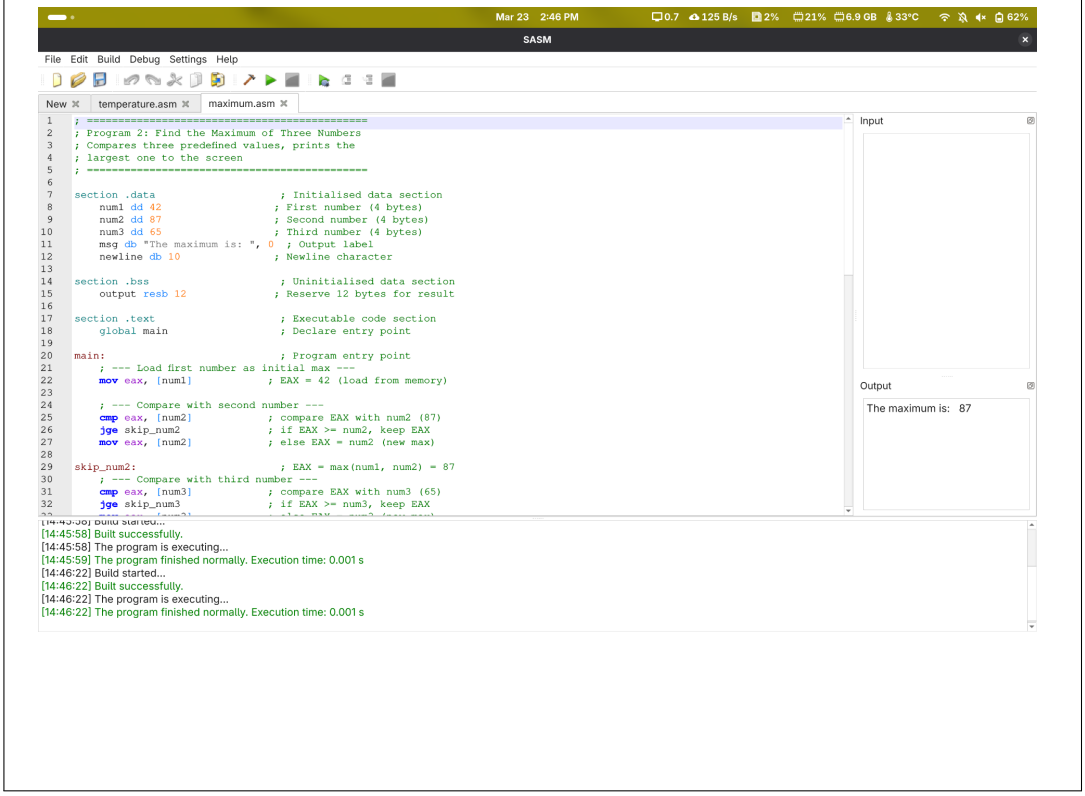
```
52      mov eax, 4                  ; sys_write
53      mov ebx, 1                  ; stdout
54      mov ecx, msg                ; pointer to label string
55      mov edx, 17                 ; length of label
56      int 0x80                    ; invoke kernel
57
58      ; --- Print the number ---
59      mov eax, 4                  ; sys_write
60      mov ebx, 1                  ; stdout
61      mov ecx, edi                ; pointer to first digit
62      mov edx, output             ; calculate string length
63      add edx, 11
64      sub edx, edi                ; EDX = number of characters
65      int 0x80                    ; invoke kernel
66
67      ; --- Print newline ---
68      mov eax, 4                  ; sys_write
69      mov ebx, 1                  ; stdout
70      mov ecx, newline            ; pointer to newline
71      mov edx, 1                  ; 1 byte
72      int 0x80                    ; invoke kernel
73
74      ; --- Exit ---
75      mov eax, 1                  ; syscall 1 = sys_exit
76      xor ebx, ebx                ; return code 0
77      int 0x80                    ; terminate program
```

Listing 2: maximum.asm — Find maximum of three numbers

5.2.3 Output Description

The program compares three predefined numbers: 42, 87, and 65. It determines the largest by comparing them sequentially using `CMP` and conditional jumps. The output is: `The maximum is: 87`. To test with different values, the numbers in the `.data` section can be changed and the program re-run.

5.2.4 Screenshot: Code + Result



5.3 Program 3: Sum of Array Elements

5.3.1 Concepts Demonstrated

Array declaration and memory layout, pointer arithmetic (advancing ESI by element size), loop counter using ECX with `DEC/JNZ`, accumulation in a register, and integer-to-ASCII conversion for display.

5.3.2 Source Code with Line-by-Line Comments

```
; ==========================================
; Program 3: Sum of Array Elements
; Calculates the sum of numbers stored in an
; array using a loop and prints the result
; ==========================================

section .data                       ; Initialised data section
    array dd 10, 20, 30, 40, 50     ; Array of 5 integers (4B each)
    array_len equ 5                 ; Number of elements in array
    msg db "Sum of array: ", 0      ; Label for output
    newline db 10                   ; Newline character (line feed)

section .bss                        ; Uninitialised data section
    output resb 12                  ; Reserve 12 bytes for result

section .text                       ; Executable code section
    global main                     ; Declare entry point for linker

main:                               ; Program entry point
    mov esi, array                  ; ESI = pointer to first element
    mov ecx, array_len              ; ECX = loop counter (5)
    xor eax, eax                    ; EAX = 0 (sum accumulator)

sum_loop:                           ; Iterate through array
    add eax, [esi]                  ; sum += current element
                                    ; [ESI] dereferences the pointer
    add esi, 4                      ; advance pointer by 4 bytes
                                    ; (each dd element is 4 bytes)
    dec ecx                         ; decrement loop counter
    jnz sum_loop                    ; if counter != 0, repeat
                                    ; (jnz = jump if not zero)

    ; --- EAX now contains 150 (10+20+30+40+50) ---

    ; --- Convert integer to ASCII string ---
    mov edi, output                 ; EDI -> output buffer
    add edi, 11                     ; position at end of buffer
    mov byte [edi], 0               ; null-terminate string

convert_sum:                        ; Extract digits right-to-left
    dec edi                         ; move pointer left 1 byte
    xor edx, edx                    ; clear EDX (needed before div)
    mov ecx, 10                     ; divisor = 10
    div ecx                         ; EAX=EAX/10, EDX=EAX%10
                                    ; quotient in EAX for next iter
                                    ; remainder = last digit
    add dl, '0'                     ; convert digit to ASCII
                                    ; e.g. 5 + 48 = 53 = '5'
    mov [edi], dl                   ; store ASCII char in buffer
```

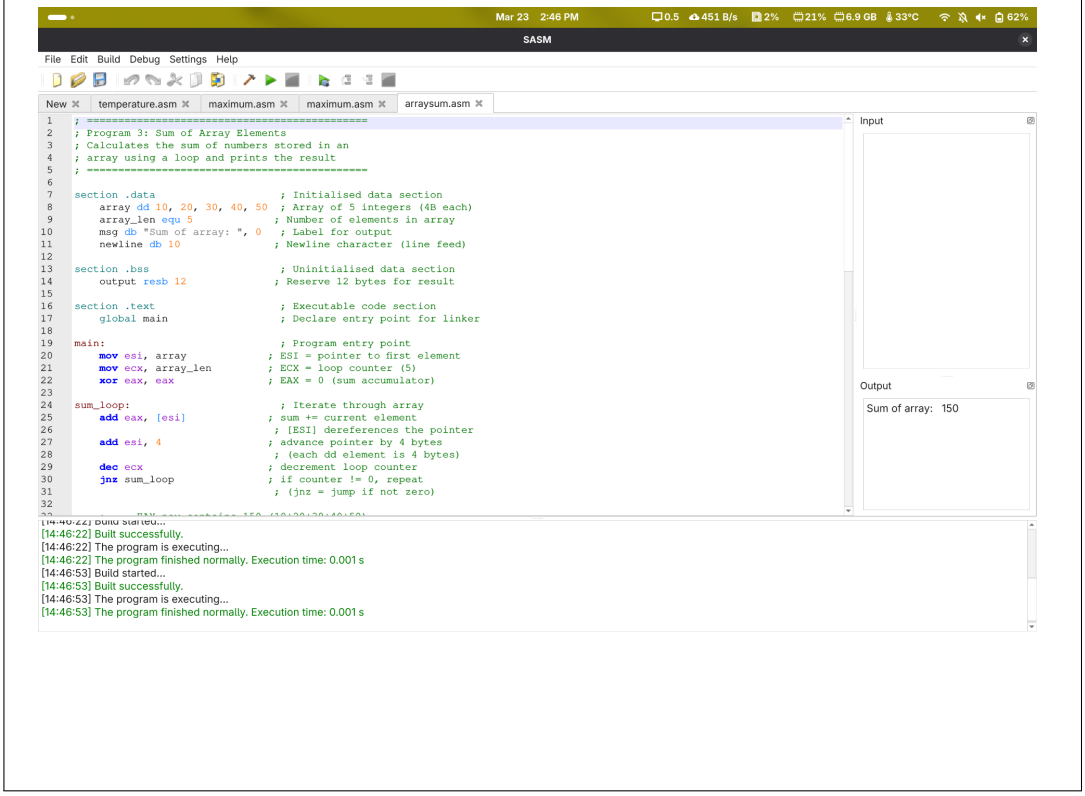
```
50      test eax, eax               ; is quotient zero?
51      jnz convert_sum             ; no -> more digits remain
52
53      ; --- Print label ---
54      mov eax, 4                  ; syscall 4 = sys_write
55      mov ebx, 1                  ; fd 1 = stdout
56      mov ecx, msg                ; pointer to label string
57      mov edx, 15                 ; number of bytes to write
58      int 0x80                    ; trigger kernel interrupt
59
60      ; --- Print the sum ---
61      mov eax, 4                  ; sys_write
62      mov ebx, 1                  ; stdout
63      mov ecx, edi                ; pointer to first digit char
64      mov edx, output             ; calculate string length:
65      add edx, 11                 ;   end of buffer position
66      sub edx, edi                ;   minus start = length
67      int 0x80                    ; print the number
68
69      ; --- Print newline ---
70      mov eax, 4                  ; sys_write
71      mov ebx, 1                  ; stdout
72      mov ecx, newline            ; pointer to newline byte
73      mov edx, 1                  ; 1 byte
74      int 0x80                    ; print it
75
76      ; --- Exit program cleanly ---
77      mov eax, 1                  ; syscall 1 = sys_exit
78      xor ebx, ebx                ; exit status 0 (no error)
79      int 0x80                    ; terminate the process
```

Listing 3: arraysum.asm — Sum of array elements

5.3.3 Output Description

The program declares an array of five integers: 10, 20, 30, 40, and 50. It iterates through the array using a loop, adding each element to a running sum in EAX. After the loop, EAX contains $10 + 20 + 30 + 40 + 50 = 150$. The result is converted to ASCII and printed: `Sum of array: 150`.

5.3.4 Screenshot: Code + Result



6 DEBUGGING

Debugging is an essential part of Assembly programming because errors at this level can be subtle and difficult to diagnose. SASM integrates with GDB (GNU Debugger) and provides visual debugging features directly in the IDE.

6.1 Key Debugging Techniques Used

Table 7: Debugging features used in SASM

Technique	**Description**
Breakpoints	Set by clicking on the line number gutter; execution pauses at each breakpoint when running in debug mode (F5)
Register monitoring	The register panel displays live values of EAX, EBX, ECX, EDX, ESI, EDI, ESP, EBP as the program executes; essential for verifying arithmetic
Step-by-step (F11)	Each instruction executes one at a time; used to trace loops and verify that counters decrement correctly
Memory viewer	Shows contents of memory addresses; used to verify that string buffers were filled correctly during integer-to-ASCII conversion

6.2 Debugging Example

While developing Program 1 (Temperature Converter), a common mistake was forgetting the `cdq` instruction before `idiv`. Without `cdq`, the EDX register contains leftover data from a previous operation, causing the division to produce an incorrect result or a floating-point exception. By setting a breakpoint at the `idiv` line and inspecting EDX in the register panel, the issue was immediately visible — EDX was not zero as expected. Adding `cdq` before `idiv` resolved the problem.

7 CONCLUSION

This laboratory exercise provided a comprehensive introduction to Assembly Language programming. Through the process of setting up the SASM development environment with NASM on Arch Linux, and writing three distinct programs, several fundamental concepts were solidified.

The three programs covered a broad range of Assembly concepts:

- **Temperature Converter** — user I/O, ASCII-to-integer and integer-to-ASCII conversion, arithmetic operations (multiplication, division, addition)
- **Maximum of Three Numbers** — memory-defined variables, comparison instruction `CMP`, conditional jumps and branching
- **Sum of Array Elements** — array traversal, pointer arithmetic, loop counter with `DEC`/`JNZ`, accumulation in a register

The debugging process proved to be one of the most educational aspects of the exercise. Being able to watch register values change in real time, set breakpoints, and step through code instruction by instruction gave a much deeper understanding of how the CPU executes programs than any high-level language debugger could provide.

Assembly Language, while challenging and verbose compared to modern languages, offers an unmatched perspective on how computers actually work. The skills gained in this laboratory form a strong foundation for understanding computer architecture, operating systems, and performance-critical software development.